Accuracy Is Not Enough

---- *Confusion Matrix Metrics That Actually Work in CVE Impact Prediction.* ----

> *by Keerthana Purushotham • 6 min read*


```
Copyright © 2025  Keerthana Purushotham
<keep.consult@proton.me>.
Licensed under the GNU AGPL v3. See LICENSE for details.
```


Cite this article -

> ***github.com/keerthanap8898/Accuracy-is-Not-Enough-in-Cybersecurity/tree/main/Cite_this_Article***
>
> ---
>
> 1. **Plain Text** :
>
> ```text/md
> Purushotham, K. (2025, September 19th). *Accuracy is not enough: Confusion matrix metrics that actually work in CVE impact prediction.* Substack. https://keerthanapurushotham.substack.com/p/accuracy-is-not-enough-confusion
>
> Also available at:
> - Medium: https://medium.com/@keerthanapurushotham/accuracy-is-not-enough-confusion-matrix-metrics-that-actually-work-in-cve-impact-prediction-d4bafd9cec1b
> - GitHub: https://github.com/keerthanap8898/Accuracy-is-Not-Enough-in-Cybersecurity/tree/main
> ```

2. **APA (7th Edition)** :

```text/md
    Purushotham, K. (2025, [September 19th]). Accuracy is not enough: Confusion matrix metrics that actually work in CVE impact prediction. [Substack - keerthanapurushotham.substack.com/p/accuracy-is-not-enough-confusion] (https://keerthanapurushotham.substack.com/p/accuracy-is-not-enough-confusion)
    Also available at:
    [Medium] (https://medium.com/@keerthanapurushotham/accuracy-is-not-enough-confusion-matrix-metrics-that-actually-work-in-cve-impact-prediction-d4bafd9cec1b)
    [GitHub](https://github.com/keerthanap8898/Accuracy-is-Not-Enough-in-Cybersecurity/tree/main)
```

3. **IEEE Style** :

```text/md
    K. Purushotham, “Accuracy Is Not Enough: Confusion Matrix Metrics That Actually Work in CVE Impact Prediction,” Substack, Sep. 2025. [Open-source on Substack: keerthanapurushotham.substack.com/p/accuracy-is-not-enough-confusion] (https://keerthanapurushotham.substack.com/p/accuracy-is-not-enough-confusion)
    Also available at:
    - [Medium] (https://medium.com/@keerthanapurushotham/accuracy-is-not-enough-confusion-matrix-metrics-that-actually-work-in-cve-impact-prediction-d4bafd9cec1b)
    - [GitHub](https://github.com/keerthanap8898/Accuracy-is-Not-Enough-in-Cybersecurity/tree/main)
```

4. **BibTeX Entry** :

```bibtex
    @misc{purushotham2025accuracy,
    author       = {Keerthana Purushotham},
    title        = {Accuracy Is Not Enough: Confusion Matrix
Metrics That Actually Work in CVE Impact Prediction},
    howpublished = {Substack},
    year         = {2025},
    month        = {September},
    url          =
{https://keerthanapurushotham.substack.com/p/accuracy-is-not-
enough-confusion},
    note         = {Also available on Medium and GitHub:

\url{https://medium.com/@keerthanapurushotham/accuracy-is-not-
enough-confusion-matrix-metrics-that-actually-work-in-cve-
impact-prediction-d4bafd9cec1b},

\url{https://github.com/keerthanap8898/Accuracy-is-Not-Enough-
in-Cybersecurity}}
                    }
```

5. **MODS xml** :

```xml
    <mods xmlns="http://www.loc.gov/mods/v3" version="3.7">
    <titleInfo>
      <title>Accuracy Is Not Enough</title>
      <subTitle>Confusion Matrix Metrics That Actually Work in
CVE Impact Prediction</subTitle>
    </titleInfo>

    <name type="personal">
      <namePart>Purushotham, Keerthana</namePart>
      <role>
        <roleTerm type="code"
authority="marcrelator">aut</roleTerm>
      </role>
    </name>
```

```
    <typeOfResource>text</typeOfResource>
    <genre>Blog post</genre>

    <originInfo>
      <publisher>Substack</publisher>
      <place>
        <placeTerm type="text">Online</placeTerm>
      </place>
      <dateIssued encoding="w3cdtf" keyDate="yes">2025-
09</dateIssued>
    </originInfo>

    <language>
      <languageTerm type="code" authority="iso639-
2b">eng</languageTerm>
    </language>

    <identifier
type="uri">https://keerthanapurushotham.substack.com/p/accuracy-
is-not-enough-confusion</identifier>

    <location>
      <url usage="primary
display">https://keerthanapurushotham.substack.com/p/accuracy-
is-not-enough-confusion</url>
    </location>

    <relatedItem type="otherVersion" displayLabel="Also
available at">
      <location>

<url>https://medium.com/@keerthanapurushotham/accuracy-is-not-
enough-confusion-matrix-metrics-that-actually-work-in-cve-
impact-prediction-d4bafd9cec1b</url>
      </location>
    </relatedItem>

    <relatedItem type="otherVersion">
      <location>
        <url>https://github.com/keerthanap8898/Accuracy-is-
Not-Enough-in-Cybersecurity</url>
      </location>
```

```
    </relatedItem>

    <note type="statement of responsibility">By Keerthana
Purushotham.</note>

    <recordInfo>
      <recordContentSource>Self-
deposited</recordContentSource>
      <recordCreationDate encoding="w3cdtf">2025-09-
19</recordCreationDate>
      <languageOfCataloging>
        <languageTerm type="code" authority="iso639-
2b">eng</languageTerm>
      </languageOfCataloging>
    </recordInfo>
  </mods>
```


🔗 **Read the full post here:**

- Medium: medium.com/post/d4bafd9cec1b
- Substack: substack.com/home/post/p-174062640
- LinkedIn: linkedin.com/posts/keerthanapurushotham_vulnerabilitymanagement-cve-security-activity-7374937046538436608-quKj

In cybersecurity,

---- **mapping vulnerabilities (CVEs) across Linux distributions is not just classification — it's risk control. Whether `python-requests` in Debian matches `python3-requests` in Red Hat can mean the difference between a patched system and an exploitable one.**

Why does this matter?

> False negatives (missed vulnerabilities) leave blind spots.
>
> False positives (over-flagged safe packages) burn developer time and erode trust.

> Accuracy hides both problems.

The solution:

- *treat the confusion matrix as a toolbox, and climb through its metrics step by step.*

Leveled Confusion Matrix Metrics Framework (L0–L6)

> Leveled Confusion Matrix Metrics Framework (L0–L6)
>
> **Level 0**
> **(Core Counts)**
> **TP, TN, FP, FN**
>
> **Level 1**
> **(Class Rates)**
> **Recall (TPR), Specificity (TNR), FNR, FPR**
>
> **Level 2**
> **(Predictive Values)**
> **Precision (PPV), NPV, FDR, FOR**
>
> **Level 3**
> **(Likelihood Ratios)**
> **PLR, NLR**
>
> **Level 4**
> **(Balanced Metrics)**
> **F1-Positive, F1-Negative, Balanced Accuracy, Youden's J**
>
> **Level 5**
> **(Global Correlation)**
> **Cohen's Kappa, MCC**
>
> **Level 6**
> **(Snapshot)**
> **Accuracy**

Trade-Offs:

> False Negatives

> The cost of not only missing the goal but causing a loss that can end in catastrophe (e.g. a 0-day CVE miss, undetected sepsis, or a satellite collision that wipes out a multi-billion-dollar asset). That's why systems lean heavily on redundancy, richer signals, and continuous error-analysis to "catch what matters most."
>
> **False Positives**
>
> Less risky, but they erode trust and waste resources. Too many false alarms = alert fatigue, extra labor, wasted cycles (e.g. trade entries falsely flagged as fraud, AV systems braking on empty roads, false debris alarms in space). Even justice systems suffer: mis-applied risk scores can harm real lives.

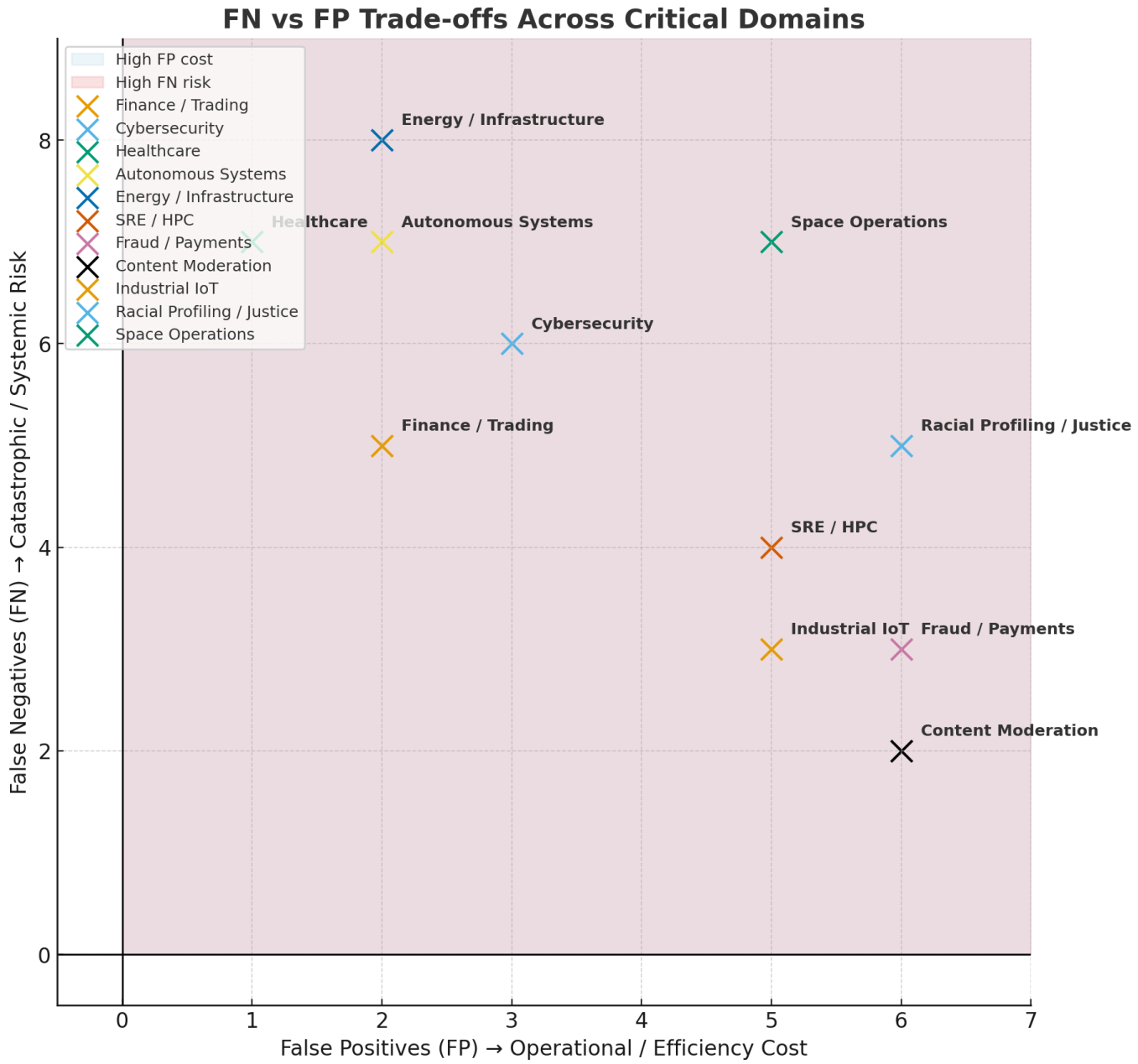
Domain Specific Examples

> 📊 **False Negatives (FN) vs False Positives (FP) Across Domains**

Domain	False Negatives (FN) → What Happens if You Miss	False Positives (FP) → What Happens if You Over-Trigger
Finance / Trading	Miss market anomaly, fraud, or risk signal → unhedged loss, regulatory fines, major capital hit.	Flagging too many trades as risky → analyst overload, slower execution, liquidity drag.
Cybersecurity	Miss zero-day exploit or critical CVE → system compromise, data breach, reputational loss.	Excess benign alerts → SOC fatigue, critical exploits buried in noise.
Healthcare / Biomed	Miss early signs of sepsis or tumor → escalation to ICU, misdiagnosis, loss of life.	Overdiagnosis (e.g., false cancer flag) → unnecessary biopsies, patient anxiety, mistrust in AI.
Autonomous Systems (Cars, Drones, Robotics)	Fail to detect pedestrian/obstacle → collisions, safety incidents, loss of life.	Over-cautious detection → unnecessary braking, route detours, traffic disruptions.
Energy / Infrastructure	Miss pipeline leak or grid anomaly → catastrophic failure, environmental damage.	Over-triggering safety shutoffs → power outages, downtime, costly maintenance calls.
SRE / HPC / Distributed Systems	Miss service degradation or corrupted state → downtime, data loss, cascading outages.	Too many false alarms → engineer burnout, wasted on-call resources, pager fatigue.

> 💡 **Pattern:**
>
> - **FN = Catastrophe risk** → financial, human, or systemic loss.
> - **FP = Trust + efficiency risk** → wasted effort, fatigue, slower throughput.

Visualization of FN vs FP tradeoffs across industries.



Level 0: Core Counts (Primitives)

The foundation of all metrics:

- **TP (True Positive):** Vulnerable package correctly flagged
- **TN (True Negative):** Safe package correctly ignored
- **FP (False Positive):** Safe package wrongly flagged
- **FN (False Negative):** Vulnerable package missed

Level 1: Class Rates (per positive/negative)

- **Recall (TPR / Sensitivity):** Prioritizes catching vulnerabilities
  - $$Recall = \frac{TP}{TP + FN}$$
- **Specificity (TNR):** Ensures safe packages aren't over-flagged
  - $$Specificity = \frac{TN}{TN + FP}$$
- Complementary rates:
  - FNR = 1 – Recall
  - FPR = 1 – Specificity

Security takeaway: FNR is the costliest mistake; FPR is the loudest one.

Level 2: Predictive Values (per outcome)

- **Precision (PPV):** Of the flagged packages, how many are truly vulnerable?
  - $$Precision = \frac{TP}{TP + FP}$$
- **Negative Predictive Value (NPV):** Of the safe-labeled packages, how many actually are safe?
  - $$NPV = \frac{TN}{TN + FN}$$

Complements:

- FDR = 1 – Precision
- FOR = 1 – NPV

Security takeaway: FOR is underused but crucial — it reveals the risk of trusting a "safe" prediction.

Level 3: Likelihood Ratios (decision-level odds)

- **Positive Likelihood Ratio (PLR):** How much a positive result increases odds of vulnerability
  - $$PLR = \frac{Recall}{FPR}$$
- **Negative Likelihood Ratio (NLR):** How much a negative result decreases odds of vulnerability
  - $$NLR = \frac{FNR}{Specificity}$$

> Security takeaway: PLR measures alert trustworthiness; NLR measures trust in "safe" decisions.

Level 4: Balanced Metrics (threshold trade-offs)

- **F1-Positive:** Balances catching vulnerabilities vs avoiding wasted alerts
  - $$F1_{+} = \frac{2 \cdot (Precision \cdot Recall)}{Precision + Recall}$$
- **F1-Negative:** Balances suppressing false alarms vs maintaining trust
  - $$F1_{-} = \frac{2 \cdot (NPV \cdot Specificity)}{NPV + Specificity}$$
- **Balanced Accuracy (BA):** Mitigates bias in imbalanced datasets

- $$BA = \frac{Recall + Specificity}{2}$$

- **Youden's J:** Quick threshold selector maximizing separation from randomness

  - $$J = Recall + Specificity - 1$$

Level 5: Agreement and Global Correlation

- **Cohen's Kappa (κ):** Corrected measure of agreement beyond chance

  - $$\kappa = \frac{p_o - p_e}{1 - p_e}$$

- **Matthews Correlation Coefficient (MCC):** Robust metric stable across imbalanced datasets; applicable in security, bioinformatics, finance

  - $$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Level 6: Baseline Snapshot

- **Accuracy:** A quick overall number — but misleading when vulnerabilities are rare

  - $$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

> Security takeaway: Accuracy is for dashboards, not critical risk decisions.

Practical Recommendations

- Lead with **Recall** and **FNR**, because misses are catastrophic

- Track **FOR** and **NLR**, because trusting “safe” labels is risky
- Use **Balanced Accuracy** and **Youden’s J** to choose thresholds
- Report **MCC** and **F1** for robustness
- Treat **Accuracy** only as a sanity check

> Both matters: FN kills you outright; FP bleeds you slowly.

Glossary of Acronyms

1. **CVE** — Common Vulnerabilities and Exposures
2. **TP** — True Positive
3. **TN** — True Negative
4. **FP** — False Positive
5. **FN** — False Negative
6. **TPR / Recall / Sensitivity** — Proportion of vulnerabilities correctly flagged
7. **TNR / Specificity** — Proportion of safe packages correctly ignored
8. **FNR** — False Negative Rate
9. **FPR** — False Positive Rate
10. **PPV / Precision** — Of flagged items, how many truly are vulnerable
11. **NPV** — Of safe-labeled items, how many truly are safe
12. **FDR** — False Discovery Rate
13. **FOR** — False Omission Rate
14. **PLR** — Positive Likelihood Ratio
15. **NLR** — Negative Likelihood Ratio
16. **BA / Balanced Accuracy** — Average of Recall and Specificity
17. **MCC** — Matthews Correlation Coefficient
18. **κ / Cohen’s Kappa** — Chance-corrected measure of agreement

Summary Cheat Sheet:

Term / Metric	Formulae	Meaning	Interpretation	Examples	Relevance	Expected Trend
Accuracy 0 to 1 (higher is better)	(TP+TN) / (TP+TN+FP+FN)	Overall proportion of correct predictions among all predictions.	High accuracy means most pkgs are correctly classified. However, it can be misleading in highly imbalanced data (few vuln vs. many safe pkgs).	Quick triage check, e.g., verifying your system accurately labels known "affected/unaffected" states for popular pkgs like nss, bash, etc.	default	increase
Positive Precision 0 to 1 (higher is better)	TP / (TP+FP)	Of all pkgs predicted vulnerable, how many truly are vulnerable?	A measure of how trustworthy a pos (vulnerable) classification is. High precision reduces false alarms.	Critical in resource-limited envs, ensures that if you flag a pkg as "affected," it is indeed correct, preventing unnecessary patching / overhead.	FP related	increase
Recall (Sensitivity, TPR) 0 to 1 (higher is better)	TP / (TP+FN)	Of all truly vulnerable pkgs, how many are identified?	High recall →fewer missed CVEs. Key in security since missing a real CVE can be catastrophic.	Key for ensuring that no known threat is overlooked (e.g., the infamous "openssl-heartbleed" scenario).	FP related	increase
Specificity (True Neg Rate) 0 to 1 (higher is better)	TN / (TN+FP)	Of all true non-vulnerable pkgs, how many are correct	High specificity ensures you do not waste time / resources patching pkgs that are truly safe.	Prevents "crying wolf" in advisories by leaving out pkgs that truly are not affected, avoiding confusion & unnecessary triage.	FP related	not greatly increase
False-pos Rate (FPR) 0 to 1 (0 is ideal)	FP / (TN+FP)	In true non-vulnerable pkgs, how many are incorrectly positive	Lower FPR means fewer false alarms.	In large-scale patch mgmt, reduces confusion & overhead from needless alerts.	FP related	not greatly increase
False-neg Rate (FNR) 0 to 1 (0 is ideal)	FN / (TP+FN)	Among truly vulnerable pkgs, how many are missed?	High FNR is particularly dangerous: CVEs remain open to potential exploits.	Missing a flagged pkg from Debian that is also relevant to Red Hat can leave systems exposed if not patched.	FN related	decrease
False Discovery Rate (FDR) 0 to 1 (lower is better)	FP / (TP+FP)	Proportion of predicted vulnerable pkgs that are truly not vulnerable.	Emphasizes how many flagged pos are false alarms	If you map thousands of pkgs across distributions, a low FDR helps ensure that your final list of "affected pkgs" is accurate.	FP related	not greatly increase
False Omission Rate (FOR) 0 to 1 (lower is better)	FN / (TN+FN)	Proportion of predicted non-vulnerable pkgs that are truly vulnerable.	(1 - Neg Predictive Value). High FOR means fq misses of true CVEs among those labeled "safe."	In official advisories, labeling a pkg "not affected" when it truly is can create significant security gaps.	FN related	decrease
F1-Pos (F1 for Pos Class) 0 to 1 (higher is better)	(2 × Precision × Recall) /(Precision + Recall)	Harmonic mean of precision & recall for vulnerable pkgs.	Useful in imbalanced data to balance capturing CVEs (recall) with minimizing FP (precision).	Captures how well your system detects real CVEs without over-flagging (ideal in scenarios where most pkgs are safe, but a few are truly vulnerable).	FN related	increase
F1-Neg (F1 for Neg Class) 0 to 1 (higher is better)	(2× NPV × specificity) / (NPV + Specificity)	Harmonic mean of specificity & neg-precision [NPV= TN /(TN+FN)]on safe pkgs.	Focuses on how accurately you identify safe pkgs, balancing specificity & correctness of neg guesses.	If you care about not wrongly labeling safe pkgs as vulnerable, F1-Neg ensures correctness on neg class.	FP related	not greatly increase
Balanced Accuracy 0 to 1 (higher is better)	(Recall+ Specificity) / 2	Avg of recall (sensitivity) & specificity.	Better than plain accuracy for imbalanced classes.	When most pkgs are safe, it ensures both detection (recall) & correct identification of safe pkgs (specificity) are measured fairly.	normalized accuracy	increase
Matthews Correlation Coefficient (MCC) +1 to -1	{(TP×TN)−(FP×FN)} / sqrt{(TP+FP)×(TP+FN)×(TN+FP)×(TN+FN)}	Measures correlation b/w predictions & actual labels.	+1 : perfect correlation, 0 : random, −1 : total disagreement. Good single-figure measure even in highly imbalanced data.	Ideal for summarizing overall classifier performance when mapping CVEs across distributions (e.g., very few vulnerable vs. many non-vulnerable).	Closeness of predictions	increase over time
Youden's Index (+1 to -1)	Sensitivity + Specificity − 1	Max effectiveness/ Best trade-off b/w recall & specificity.	1 means no false-pos/false-negs. Closer to 1 → more ability to avoid both types.	Straightforward for gauging how well you find real CVEs (high recall) while also avoiding false-pos (high specificity).	FN related	increase
Pos Likelihood Ratio (PLR) 0 to ∞ (higher is better)	Recall / FPR	Chance of a right threat prediction wrt a false alarm.	PLR > 1 → test/model is useful for ruling in threat. The higher, the stronger the evidence of a real bug.	If a pkg is flagged as "vulnerable," PLR indicates how much more likely it is truly vulnerable compared to being a false alarm.	FP related	increase
Neg Likelihood Ratio (NLR) 0 to ∞ (lower is better)	FNR / Specificity	Chance of missing a true threat (FN) relative to correctly o/p safe pkgs (TN).	NLR<1→test/model is useful for ruling out threat. Low NLR means it's unlikely that a pkg that's "unaffected" is truly buggy.	Ensures that when you say "unaffected," you are rarely missing a genuine threat.	FN related;	decrease
Cohen's Kappa (+1 to -1)	[Observed Agreement − Expected Agreement] / [1−Expected Agreement]	Measures how much better the model is than random guessing.	+1 = perfect agreement beyond chance, 0 = purely random performance.	Helps confirm that "affected" vs. "unaffected" predictions are not simply occurring by chance in large-scale data sets.	n/a	Not monotonic

They're all metrics that are easy to map with ingestion tools too — niche statistics you'd commonly see around ML models but mathematically speaking, they apply to any statistically, predictive black box.

For example,

CVE-2023–48022 is an epitome of how a disputed CVE turned into a false-negative, hiding an actively exploited bug downstream. Tracking False-Negative-Rate (FNR) or Negative-Likelihood-Ratio (NLR) would've flagged very early on that "safe/unaffected" wasn't safe. Even MCC would've exposed the imbalance of missing rare but critical vulnerabilities.

- The JSON for OSV template has attributes that can be automated,
  - CNA [disputed] → implying no urgent fix needed.
  - CISA [affected] → treat as dangerous.

> - CVE-2023–48022 : github.com/CVEProject/cvelistV5/blob/main/cves/2023/48xxx/CVE-2023-48022.json

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

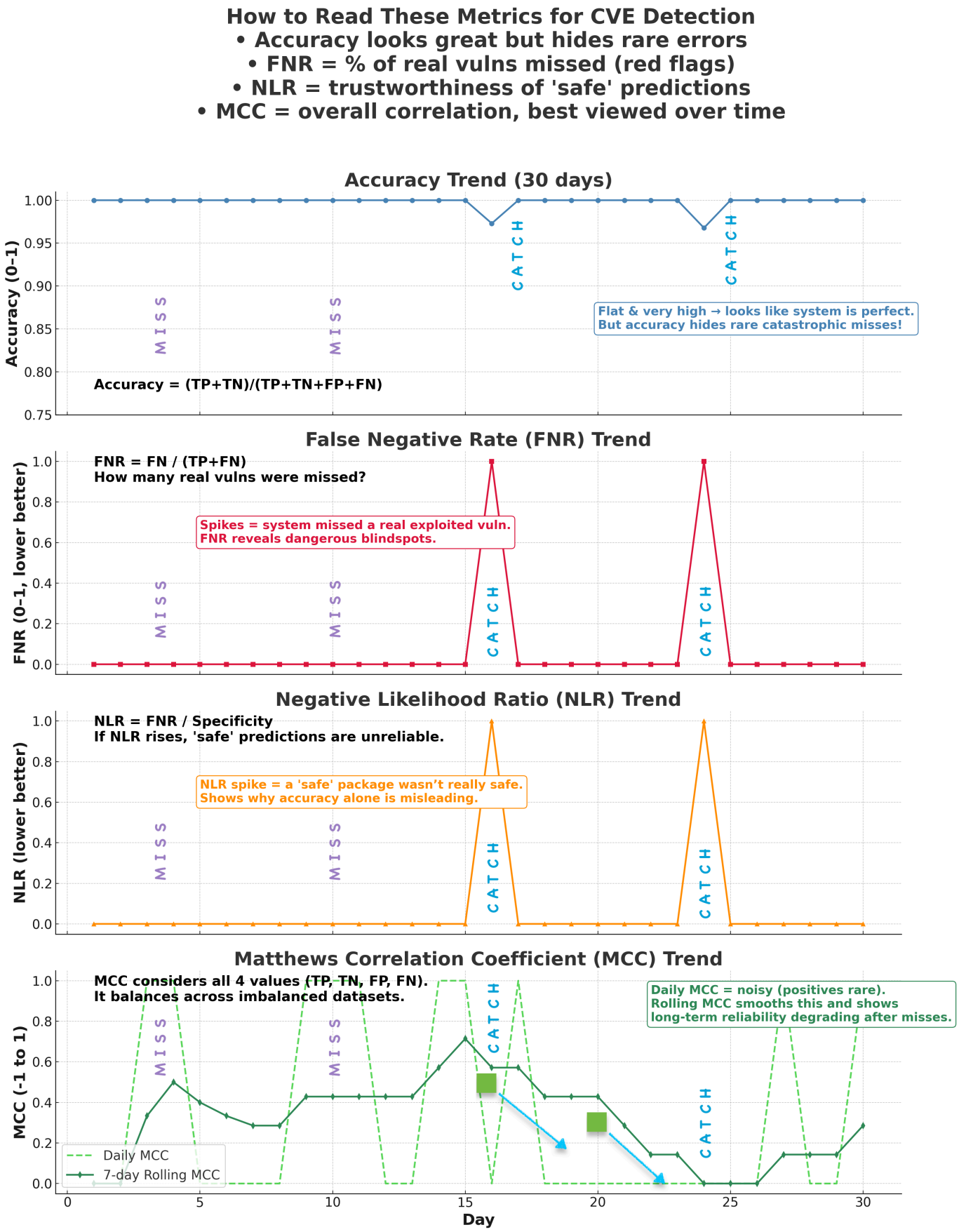

```
dataType:                    "CVE_RECORD"
cveMetadata:
  state:                     "PUBLISHED"
  cveId:                     "CVE-2023-48022"
  assignerOrgId:             "8254265b-2729-46b6-b9e3-3dfca2d5bfca"
  assignerShortName:         "mitre"
  dateUpdated:               "2024-10-28T16:10:47.152927"
  dateReserved:              "2023-11-13T00:00:00"
  datePublished:             "2023-11-28T00:00:00"
containers:
  cna:
    providerMetadata:        {…}
    descriptions:            […]
    tags:
      0:                     "disputed"
    affected:
      0:
        vendor:              "n/a"
        product:             "n/a"
        versions:
          0:
            version:         "n/a"
            status:          "affected"
    references:              […]
    problemTypes:            […]
  adp:
    0:
      problemTypes:
        0:                   {…}
      affected:
        0:                   {…}
        1:
          vendor:            "anyscale"
          product:           "ray"
          cpes:              […]
          defaultStatus:     "unknown"
          versions:
            0:
              version:       "2.8.0"
              status:        "affected"
      metrics:               […]
      title:                 "CISA ADP Vulnrichment"
      providerMetadata:      {…}
    1:                       {…}
dataVersion:                 "5.1"
```

That’s the FN suppressed incorrectly when it was being actively exploited.

- FNR (↑) spikes to 1 if an exploited vulnerability is missed.
- NLR (↑) the “safe/unaffected” label was untrustworthy.
- MCC (↓) correlation drops since a rare but critical positive was missed.

To help visualize the dashboard, I plotted a sample and it would look something like this — image attached below.

All this obviously depends on the timing of & if exploitations/bugs in your tool are identifiable.



The blue arrows along the MCC graph indicates decrease in overall prediction-mapping trust.

Closing

Security/threat impact prediction isn't about being “mostly right.” It’s about being “right where it counts.”

Thus, this leveled framework transforms the confusion matrix into a decision map — one that uncovers blind spots, tames operational noise, and aligns predictions with real-world risk.

- Follow me on LinkedIn - linkedin.com/in/keerthanapurushotham for more work.
- Email: keep.consult@proton.me

- Cite this Article: github.com/keerthanap8898/Accuracy-is-Not-Enough-in-Cybersecurity/tree/main/Cite_this_Article